Practical Work Report

TP05

Real-Time Data Streaming and Processing Using Apache Kafka

Module: Big Data Analytics

BenAttous NourEddine G2

Prepared by:
Master 2 AI & Data Science Student

University of El Oued
November 2026

Abstract

This practical work explores real-time data streaming and processing using Apache Kafka. The objective is to understand the fundamental concepts of distributed event streaming and apply them through a simulated campus sensor monitoring system.

The implementation uses a Python producer to generate temperature and humidity events and publish them to a Kafka topic. Python consumers subscribe to the topic and process the incoming messages. Kafka is deployed locally using Docker Compose, while the communication between Python applications and the Kafka broker is handled by the `kafka-python` library.

Several experiments are conducted to study topic partitioning, consumer groups, message distribution, running average calculation, and event filtering based on campus buildings.

The results demonstrate how Apache Kafka supports asynchronous communication, parallel consumption, and basic real-time stream processing.

Keywords: Apache Kafka, Big Data, Data Streaming, Producer, Consumer, Consumer Groups, Partitions, Python, Docker.

Acknowledgements

I would like to express my appreciation to the teaching staff of the Department of Computer Science at the University of El Oued for providing the academic framework and guidance necessary to complete this practical work.

Contents

Abstract i

Acknowledgements ii

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement . . . 1

1.3 Objectives . . . 2

2 Technologies and System Architecture 3

2.1 Technologies Used . . . 3

2.2 Apache Kafka Concepts . . . 3

2.2.1 Producer . . . 3

2.2.2 Consumer . . . 4

2.2.3 Broker . . . 4

2.2.4 Topic . . . 4

2.2.5 Partition . . . 4

2.2.6 Offset . . . 4

2.2.7 Consumer Group . . . 4

2.3 System Architecture . . . 4

2.4 Project Structure . . . 5

3 Implementation and Configuration 6

3.1 Kafka Deployment Using Docker Compose . . . 6

3.2 Kafka Topic Creation 6
3.3 Python Producer Implementation 7
3.4 Python Consumer Implementation 9

4 Experiments and Results **13**
4.1 Experiment 1: Running Average Temperature 13
4.2 Experiment 2: Consumers in the Same Consumer Group 14
4.3 Experiment 3: Different Consumer Groups 15
4.4 Experiment 4: One Partition and Two Consumers 16
4.5 Experiment 5: Three Partitions and Two Consumers 17
4.6 Experiment 6: Filtering Events by Campus Building 18

5 Discussion **21**
5.1 Analysis of the Results 21
5.2 Technical Challenges 21
5.3 Limitations 22

6 Theoretical Questions and Answers **23**
6.1 Question 1: What problem does Apache Kafka solve in a data-streaming architecture? 23
6.2 Question 2: Define producer, consumer, broker, topic, partition, offset, and consumer group. 23
6.3 Question 3: What is the difference between a Kafka topic and a partition? 24
6.4 Question 4: Why does the consumer use auto_offset_reset='earliest'? When does this setting apply? 24
6.5 Question 5: What is the purpose of the consumer group ID? What happens if a new group ID is used? 24
6.6 Question 6: Why is JSON a suitable format for this exercise? Give one limitation. 25
6.7 Question 7: Explain how Docker Compose simplifies deployment compared with starting services manually. 25
6.8 Question 8: What are the limitations of this single-broker educational setup compared with a production Kafka cluster? 25

6.9 Question 9: Attach evidence showing the running container, topic description, producer output, and consumer output. 26

7 Conclusion and Future Work **27**

7.1 Conclusion 27

7.2 Future Work 27

List of Figures

3.1 Kafka broker running inside Docker. 7

3.2 Description of the campus-sensors topic and its partitions. 7

3.3 Initial producer execution. 8

3.4 Producer sending 100 sensor events. 8

3.5 Continuation of the producer output showing additional events. 9

3.6 Consumer receiving and displaying sensor events. 10

3.7 Kafka consumer group information. 10

3.8 Consumer processing the generated events. 11

3.9 Continuation of the consumer output. 12

4.1 Running average temperature and high-temperature alerts. 14

4.2 Message distribution between two consumers in the same group. 15

4.3 Two different consumer groups receiving the same messages. 16

4.4 One partition assigned to one consumer within the group. 17

4.5 Three partitions distributed between two consumers. 18

4.6 Producer generating events for different campus buildings. 19

4.7 Consumer filtering events from building-A. 20

Chapter 1

Introduction

1.1 Background

Modern applications continuously generate large volumes of data through sensors, mobile devices, websites, and distributed services. Processing this information as it arrives is essential in many domains, including monitoring systems, financial applications, Internet of Things (IoT), and industrial automation.

Traditional batch-processing approaches are not always suitable when applications require immediate responses to incoming events. Real-time data streaming technologies address this challenge by allowing applications to publish, transport, and process data continuously.

Apache Kafka is a distributed event-streaming platform designed to handle high volumes of data reliably and efficiently.

1.2 Problem Statement

A campus monitoring system may contain multiple sensors installed in different buildings. These sensors continuously generate measurements such as temperature and humidity.

The main challenge is to collect these events, distribute them between processing applications, and extract useful information without manually transferring data between services.

This practical work addresses this problem by implementing a simulated sensor monitoring architecture using Apache Kafka.

1.3 Objectives

The main objectives of this practical work are:

- Understand the architecture and fundamental concepts of Apache Kafka.
- Deploy a Kafka broker using Docker Compose.
- Create topics and configure partitions.
- Implement Python producers and consumers.
- Publish and consume JSON messages.
- Understand consumer groups and partition assignment.
- Calculate a running average temperature.
- Filter events according to campus building identifiers.

Chapter 2

Technologies and System Architecture

2.1 Technologies Used

The project was implemented using the following technologies:

Table 2.1: Technologies used in the project

Technology	**Purpose**
Apache Kafka 3.9.1	Distributed event streaming platform
Docker	Containerization
Docker Compose	Service deployment and configuration
Python	Producer and consumer implementation
kafka-python 2.0.2	Python Kafka client library
JSON	Structured event representation
PowerShell	Command-line execution

2.2 Apache Kafka Concepts

2.2.1 Producer

A producer is an application that publishes messages to Kafka topics. In this project, the producer generates simulated sensor events and sends them to the `campus-sensors` topic.

2.2.2 Consumer

A consumer subscribes to a Kafka topic and retrieves its messages for processing. The consumer implemented in this project processes sensor readings and calculates their running average.

2.2.3 Broker

A broker is a Kafka server responsible for receiving, storing, and delivering messages. Our implementation uses a single Kafka broker running inside a Docker container.

2.2.4 Topic

A topic is a logical category used to organize messages. The main topic used in this project is:

`campus-sensors`

2.2.5 Partition

A partition is an ordered sequence of messages within a topic. Partitions allow Kafka to distribute messages and support parallel consumption.

The main topic was configured with three partitions.

2.2.6 Offset

An offset is a sequential identifier assigned to each message within a partition. Consumers use offsets to track their reading positions.

2.2.7 Consumer Group

A consumer group is a set of consumers that cooperate to read messages from a topic. Kafka distributes partitions among the consumers belonging to the same group.

2.3 System Architecture

The implemented system follows this architecture:

Simulated Campus Sensors

↓

Python Producer

↓

Apache Kafka Broker

↓

campus-sensors Topic

↓

Python Consumers

↓

Real-Time Data Processing

The producer generates events and publishes them to Kafka. The broker stores the messages in topic partitions. Consumers subscribe to the topic and retrieve the events for processing.

2.4 Project Structure

The project directory contains the following files:

```
tp05-kafka/
|
|-- screenshots/
|-- compose.yaml
|-- requirements.txt
|-- producer.py
|-- consumer.py
|-- consumer_groups_test.py
|-- partition_test_consumer.py
|-- .gitignore
|-- README.md
```

Chapter 3

Implementation and Configuration

3.1 Kafka Deployment Using Docker Compose

Kafka was deployed using Docker Compose to simplify the configuration and execution of the broker.

The `compose.yaml` file defines the Kafka image, network ports, broker configuration, persistent volume, and restart policy.

The following command starts the Kafka service:

```
docker compose up -d
```

The running container was verified using:

```
docker ps
```

3.2 Kafka Topic Creation

The main topic was created with three partitions and a replication factor of one.

```
docker exec -it tp05-kafka /opt/kafka/bin/kafka-topics.sh \
--bootstrap-server localhost:9092 \
--create --if-not-exists \
--topic campus-sensors \
--partitions 3 \
--replication-factor 1
```

The topic configuration was inspected using:

```
docker exec -it tp05-kafka /opt/kafka/bin/kafka-topics.sh \
```

```
--bootstrap-server localhost:9092 \
--describe --topic campus-sensors
```

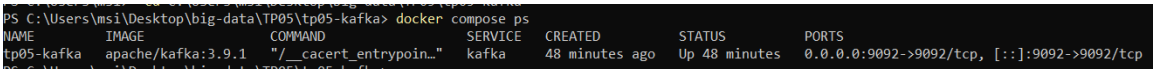


Figure 3.1: Kafka broker running inside Docker.

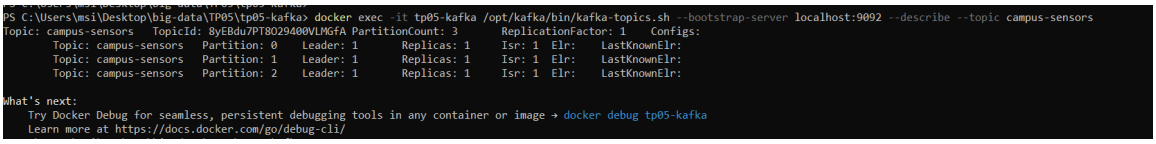


Figure 3.2: Description of the campus-sensors topic and its partitions.

3.3 Python Producer Implementation

The producer is implemented in `producer.py`. It generates simulated sensor measurements and publishes them to the Kafka topic.

Each event contains the following fields:

- `event_id`: Unique event identifier.
- `sensor_id`: Sensor identifier.
- `campus_building`: Building identifier.
- `temperature_c`: Temperature in Celsius.
- `humidity_pct`: Relative humidity percentage.
- `timestamp`: Event generation time.

The events are serialized into JSON before being sent to Kafka.

The producer uses the `acks="all"` configuration to request acknowledgement from the required in-sync replicas. In this single-broker setup, the replication factor is one.

```
producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda value:
        json.dumps(value).encode("utf-8"),
    acks="all"
)
```

The producer was configured to generate 100 events, with random temperature, humidity, sensor, and building values.

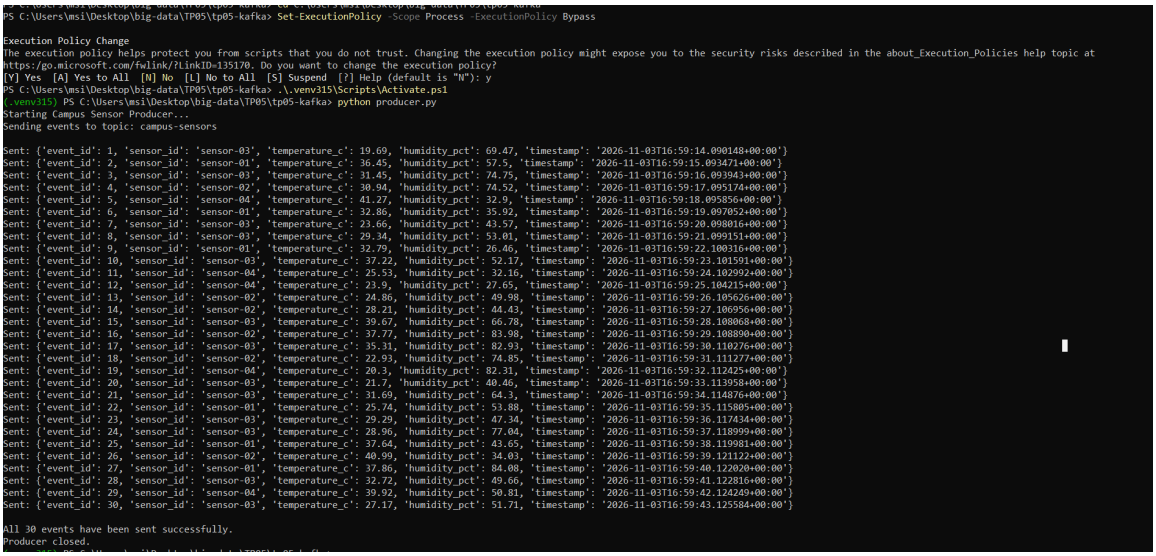


Figure 3.3: Initial producer execution.

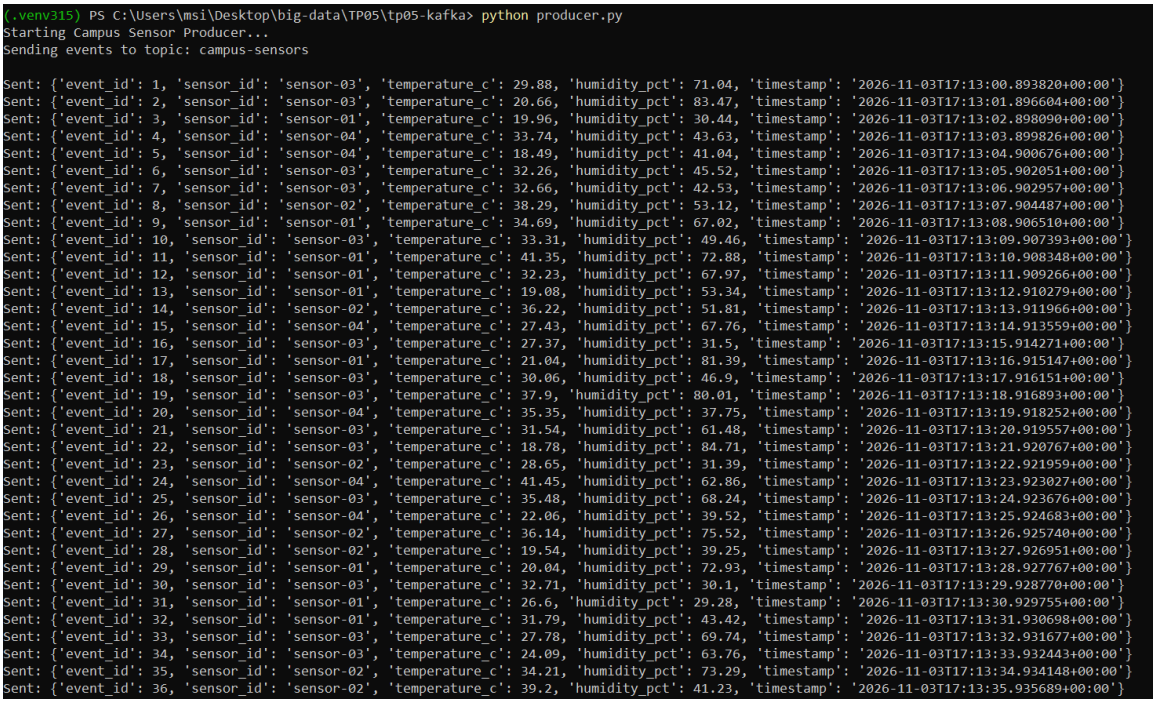


Figure 3.4: Producer sending 100 sensor events.

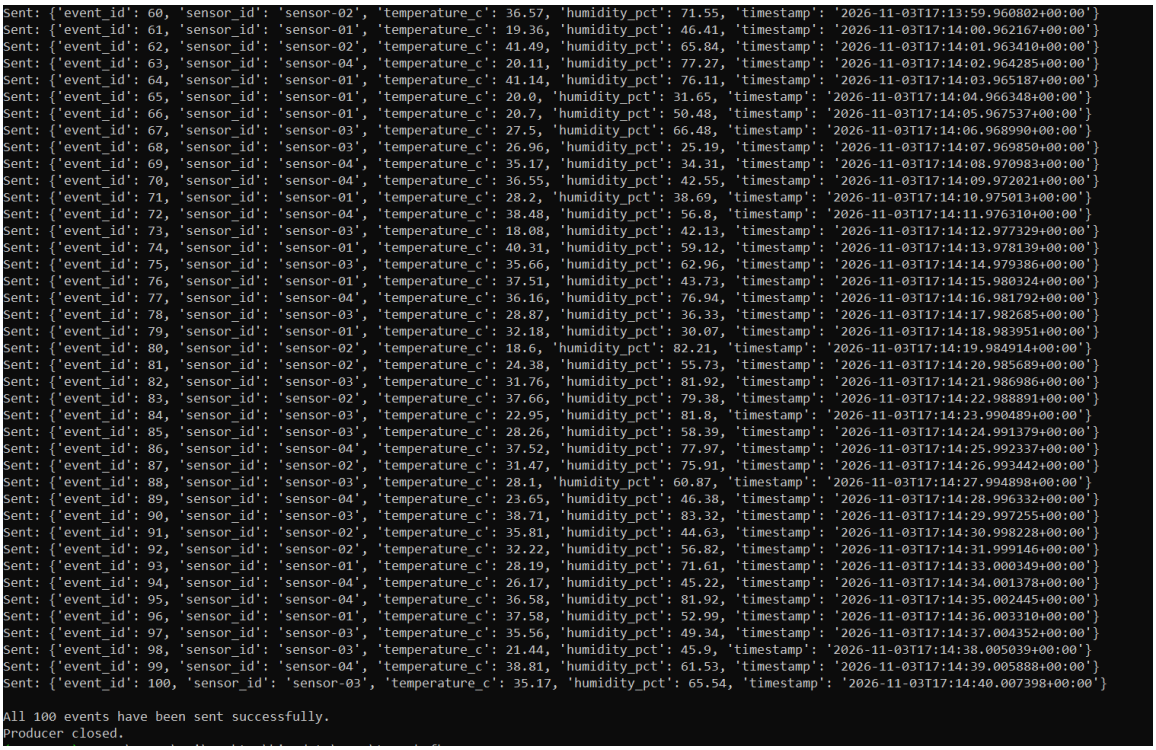


Figure 3.5: Continuation of the producer output showing additional events.

3.4 Python Consumer Implementation

The consumer is implemented in `consumer.py`. It subscribes to the `campus-sensors` topic and retrieves the published events.

The consumer uses the following configuration:

```
consumer = KafkaConsumer(
    "campus-sensors",
    bootstrap_servers="localhost:9092",
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    group_id="tp05-building-filter",
    value_deserializer=lambda value:
        json.loads(value.decode("utf-8"))
)
```

The `auto_offset_reset="earliest"` setting allows a new consumer group without a valid committed offset to start reading from the earliest available message.

The consumer processes incoming sensor events and displays their identifiers, temperature, and humidity.

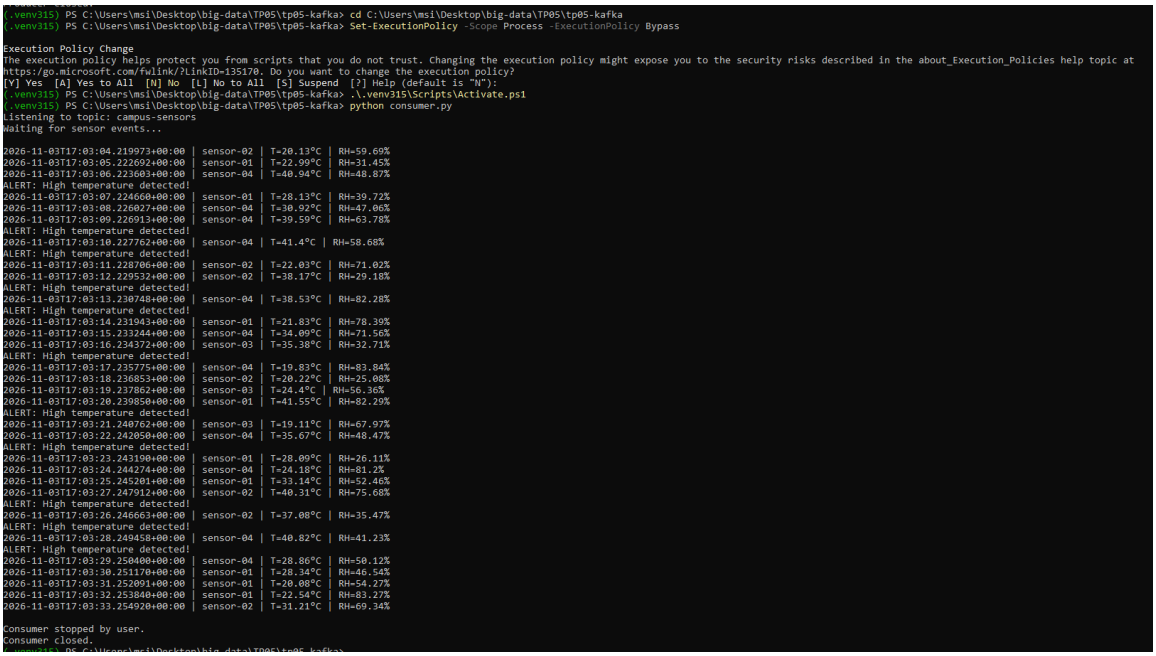


Figure 3.6: Consumer receiving and displaying sensor events.

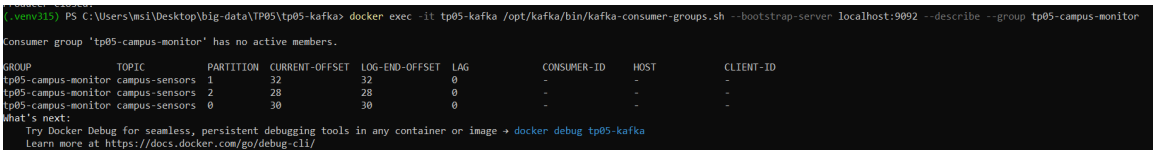


Figure 3.7: Kafka consumer group information.

```
(.venv315) PS C:\Users\msi\Desktop\big-data\TP05\tp05-kafka>  python consumer.py
Listening to topic: campus-sensors
Waiting for sensor events...

2026-11-03T17:13:00.893820+00:00 | sensor-03 | T=29.88°C | RH=71.04%
2026-11-03T17:13:01.896604+00:00 | sensor-03 | T=20.66°C | RH=83.47%
2026-11-03T17:13:02.898090+00:00 | sensor-01 | T=19.96°C | RH=30.44%
2026-11-03T17:13:03.899826+00:00 | sensor-04 | T=33.74°C | RH=43.63%
2026-11-03T17:13:04.900676+00:00 | sensor-04 | T=18.49°C | RH=41.04%
2026-11-03T17:13:05.902051+00:00 | sensor-03 | T=32.26°C | RH=45.52%
2026-11-03T17:13:06.902957+00:00 | sensor-03 | T=32.66°C | RH=42.53%
2026-11-03T17:13:07.904487+00:00 | sensor-02 | T=38.29°C | RH=53.12%
ALERT: High temperature detected!
2026-11-03T17:13:08.906510+00:00 | sensor-01 | T=34.69°C | RH=67.02%
2026-11-03T17:13:09.907393+00:00 | sensor-03 | T=33.31°C | RH=49.46%
2026-11-03T17:13:10.908348+00:00 | sensor-01 | T=41.35°C | RH=72.88%
ALERT: High temperature detected!
2026-11-03T17:13:11.909266+00:00 | sensor-01 | T=32.23°C | RH=67.97%
2026-11-03T17:13:12.910279+00:00 | sensor-01 | T=19.08°C | RH=53.34%
2026-11-03T17:13:13.911966+00:00 | sensor-02 | T=36.22°C | RH=51.81%
ALERT: High temperature detected!
2026-11-03T17:13:14.913559+00:00 | sensor-04 | T=27.43°C | RH=67.76%
2026-11-03T17:13:15.914271+00:00 | sensor-03 | T=27.37°C | RH=31.5%
2026-11-03T17:13:16.915147+00:00 | sensor-01 | T=21.04°C | RH=81.39%
2026-11-03T17:13:17.916151+00:00 | sensor-03 | T=30.06°C | RH=46.9%
2026-11-03T17:13:18.916893+00:00 | sensor-03 | T=37.9°C | RH=80.01%
ALERT: High temperature detected!
2026-11-03T17:13:19.918252+00:00 | sensor-04 | T=35.35°C | RH=37.75%
ALERT: High temperature detected!
2026-11-03T17:13:20.919557+00:00 | sensor-03 | T=31.54°C | RH=61.48%
2026-11-03T17:13:21.920767+00:00 | sensor-03 | T=18.78°C | RH=84.71%
2026-11-03T17:13:22.921959+00:00 | sensor-02 | T=28.65°C | RH=31.39%
2026-11-03T17:13:23.923027+00:00 | sensor-04 | T=41.45°C | RH=62.86%
ALERT: High temperature detected!
2026-11-03T17:13:24.923676+00:00 | sensor-03 | T=35.48°C | RH=68.24%
ALERT: High temperature detected!
2026-11-03T17:13:25.924683+00:00 | sensor-04 | T=22.06°C | RH=39.52%
2026-11-03T17:13:26.925740+00:00 | sensor-02 | T=36.14°C | RH=75.52%
ALERT: High temperature detected!
2026-11-03T17:13:27.926951+00:00 | sensor-02 | T=19.54°C | RH=39.25%
2026-11-03T17:13:28.927767+00:00 | sensor-01 | T=20.04°C | RH=72.93%
2026-11-03T17:13:29.928770+00:00 | sensor-03 | T=32.71°C | RH=30.1%
2026-11-03T17:13:30.929755+00:00 | sensor-01 | T=26.6°C | RH=29.28%
2026-11-03T17:13:31.930698+00:00 | sensor-01 | T=31.79°C | RH=43.42%
```

Figure 3.8: Consumer processing the generated events.

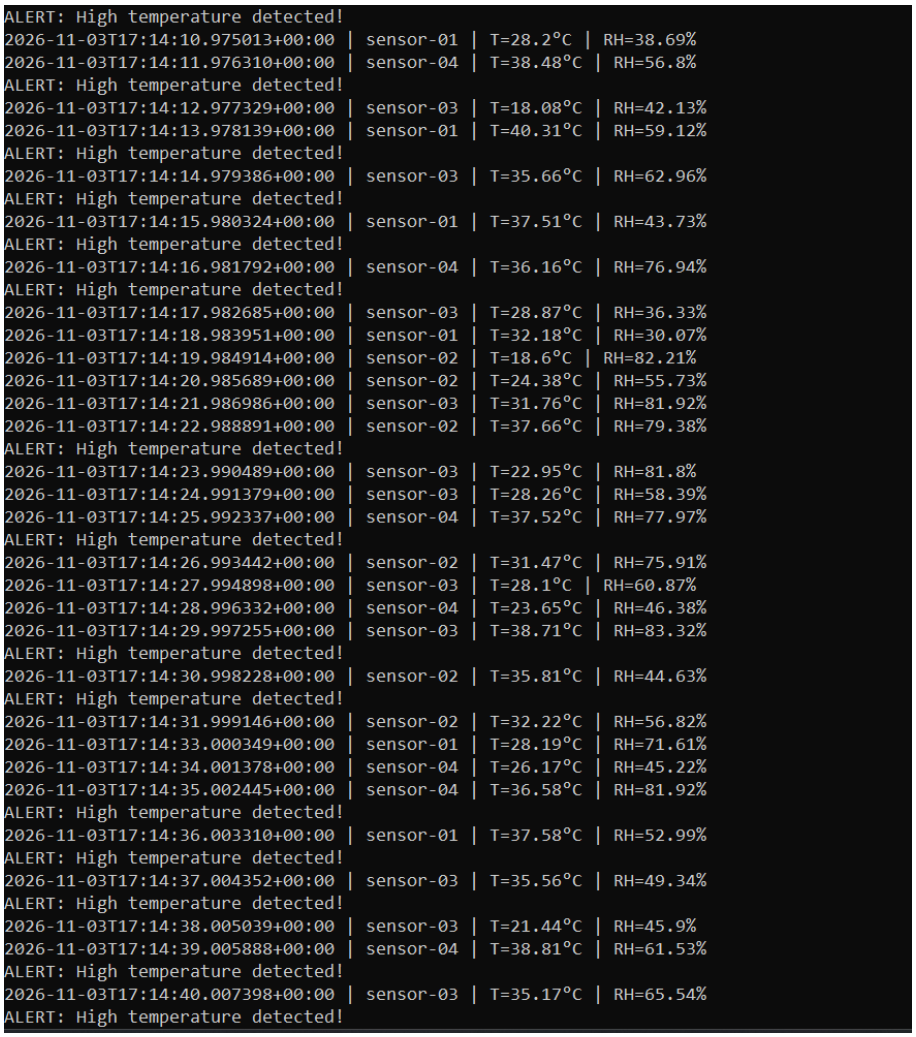


Figure 3.9: Continuation of the consumer output.

Chapter 4

Experiments and Results

4.1 Experiment 1: Running Average Temperature

The consumer was extended to calculate the running average temperature of the received sensor events.

Two variables are used to maintain the accumulated temperature and the number of processed events.

```
total_temperature = 0
event_count = 0

event_count += 1
total_temperature += temperature

average_temperature = (
    total_temperature / event_count
)
```

For every new event, the average is updated using the accumulated temperature divided by the number of processed events.

The consumer also checks whether the temperature reaches or exceeds 35 degrees Celsius. If this condition is satisfied, a high-temperature alert is displayed.

```
if temperature >= 35:
    print("ALERT: High temperature detected!")
```

Result:

The consumer successfully calculated and displayed the running average temperature while processing the incoming messages. High-temperature alerts were also generated

when the threshold was reached.

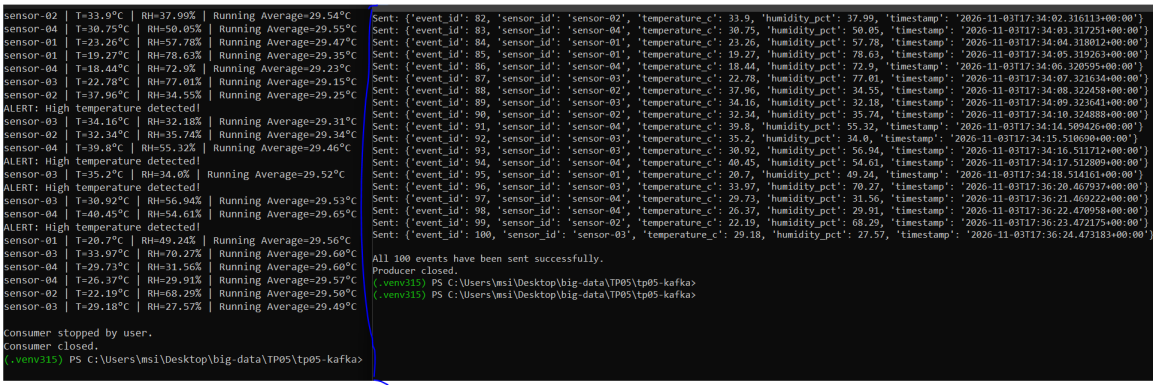


Figure 4.1: Running average temperature and high-temperature alerts.

4.2 Experiment 2: Consumers in the Same Consumer Group

In this experiment, two consumer instances were started using the same group identifier:

```
python consumer_groups_test.py shared-group
```

The two consumers subscribed to the same topic and belonged to the same consumer group.

Since the topic contains three partitions, Kafka assigned partitions between the two consumers.

The experiment demonstrated that consumers in the same group share the workload rather than independently receiving every message.

Result:

The messages were distributed between the two consumers according to their assigned partitions. One consumer received messages from two partitions, while the other received messages from the remaining partition.

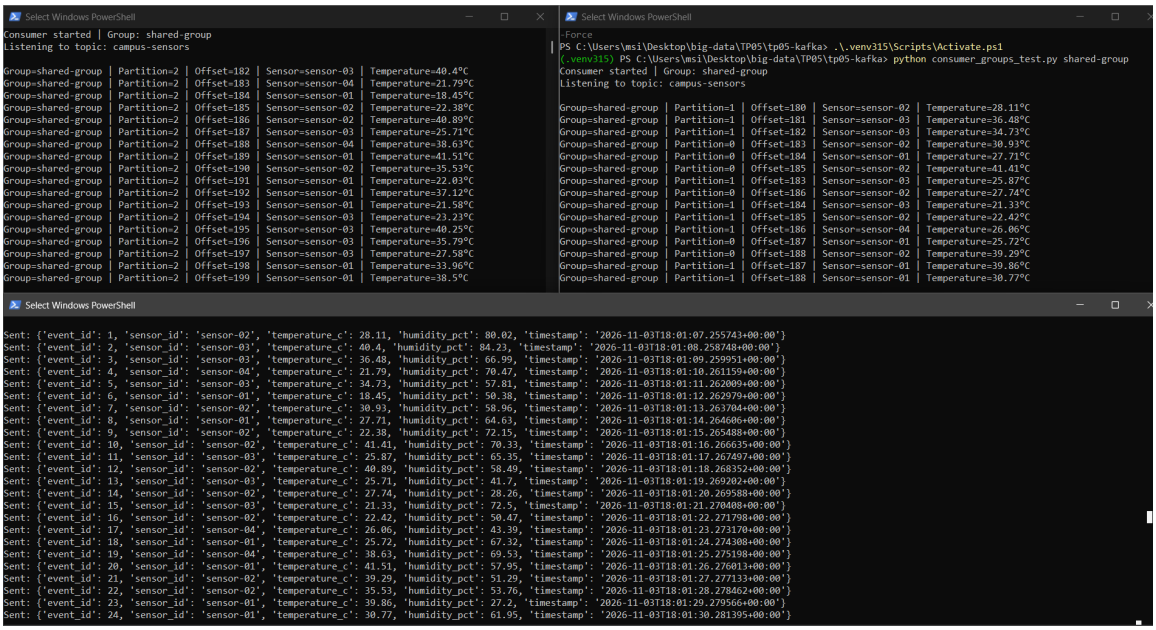


Figure 4.2: Message distribution between two consumers in the same group.

4.3 Experiment 3: Different Consumer Groups

The experiment was repeated using two different consumer group identifiers:

```
python consumer_groups_test.py group-alpha
python consumer_groups_test.py group-beta
```

Each group maintains its own independent offset information. Consequently, both groups can independently consume the same messages from a Kafka topic.

Result:

Both consumer groups successfully received the same test messages. This confirmed that different consumer groups consume topic data independently.

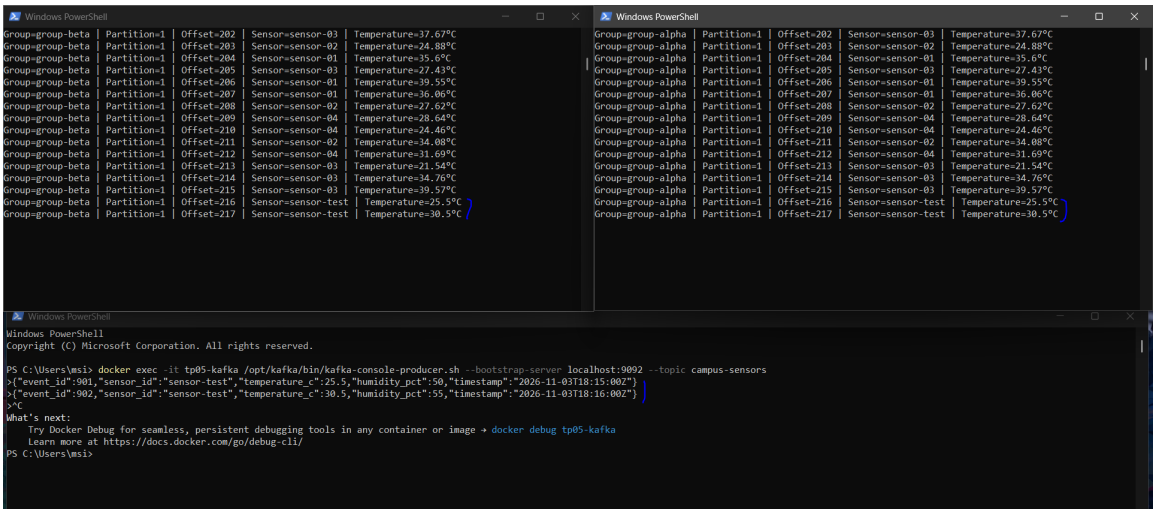


Figure 4.3: Two different consumer groups receiving the same messages.

4.4 Experiment 4: One Partition and Two Consumers

A separate topic named `campus-sensors-test` was created with only one partition.

```
docker exec -it tp05-kafka \
/opt/kafka/bin/kafka-topics.sh \
--bootstrap-server localhost:9092 \
--create --if-not-exists \
--topic campus-sensors-test \
--partitions 1 \
--replication-factor 1
```

Two consumer instances were started using the same group ID: `single-partition-group`.

Result:

Only one consumer received the messages. The second consumer remained idle because Kafka assigns each partition to only one consumer within a consumer group at a time.

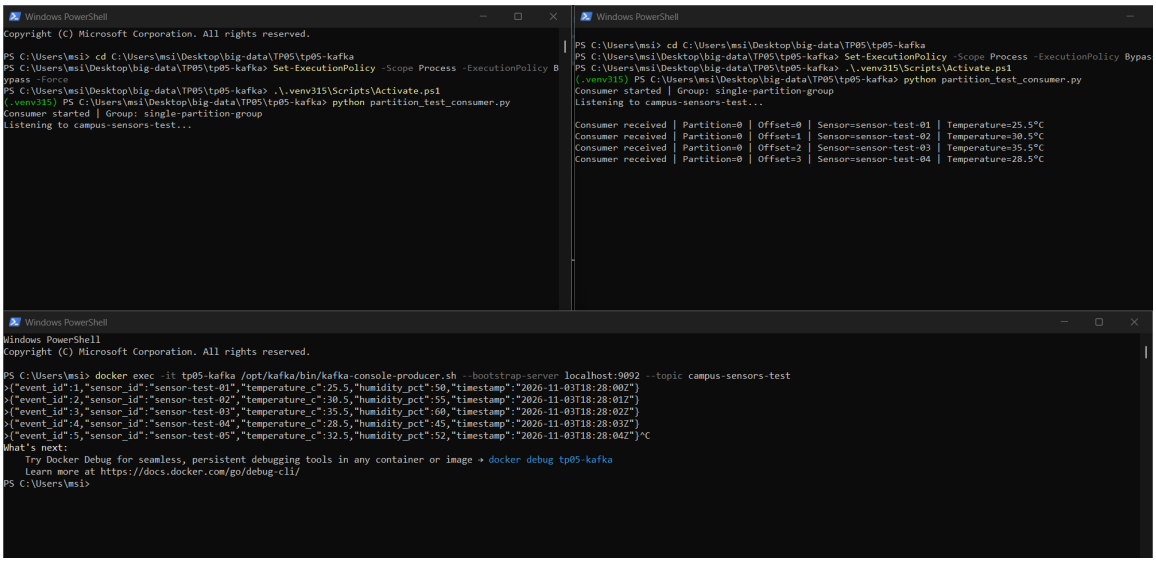


Figure 4.4: One partition assigned to one consumer within the group.

4.5 Experiment 5: Three Partitions and Two Consumers

The previous experiment was repeated using the main `campus-sensors` topic, which contains three partitions.

Two consumers were started using the same group identifier.

Result:

Kafka distributed the three partitions between the two consumers. One consumer received messages from two partitions, while the other received messages from one partition.

This experiment demonstrated how multiple partitions allow parallel consumption within a consumer group.

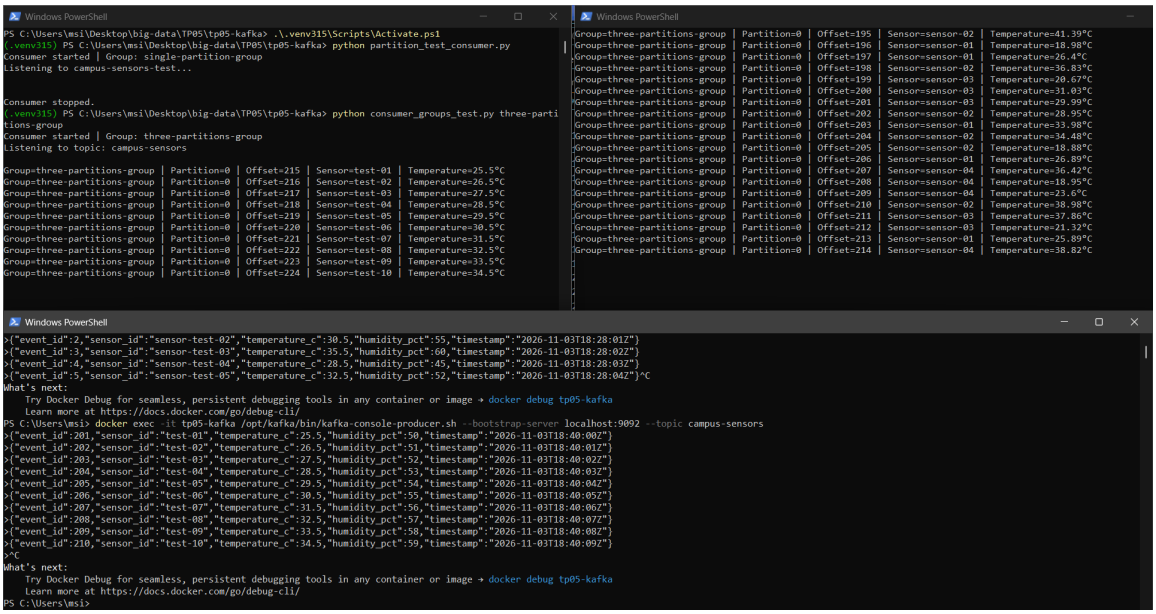


Figure 4.5: Three partitions distributed between two consumers.

4.6 Experiment 6: Filtering Events by Campus Building

The producer was modified to include a new field named `campus_building`.

The building value is selected randomly from the following identifiers:

- `building-A`
- `building-B`
- `building-C`

The consumer was then modified to process only events belonging to `building-A`.

```
TARGET_BUILDING = "building-A"

building = event.get(
    "campus_building", "unknown"
)

if building != TARGET_BUILDING:
    continue
```

Events from other buildings are ignored, while events from the selected building are processed normally.

The running average temperature is calculated using only the accepted events.

Result:

The producer successfully generated events containing different campus building identifiers. The consumer displayed only events associated with `building-A`.

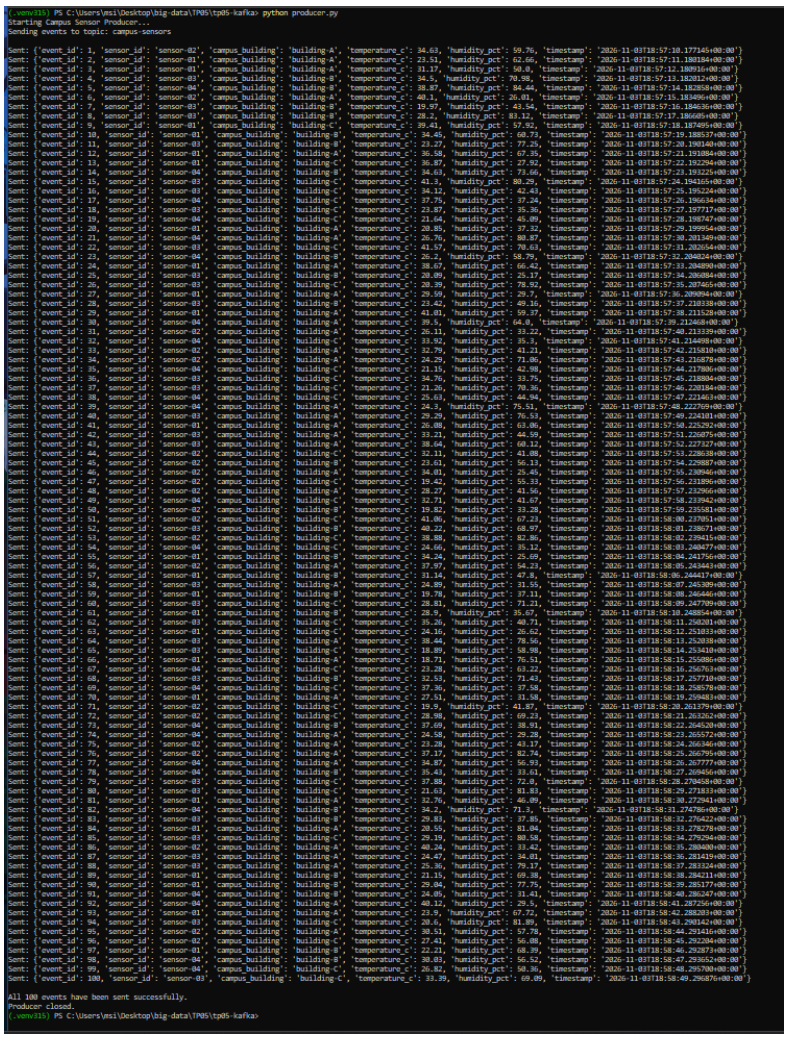


Figure 4.6: Producer generating events for different campus buildings.

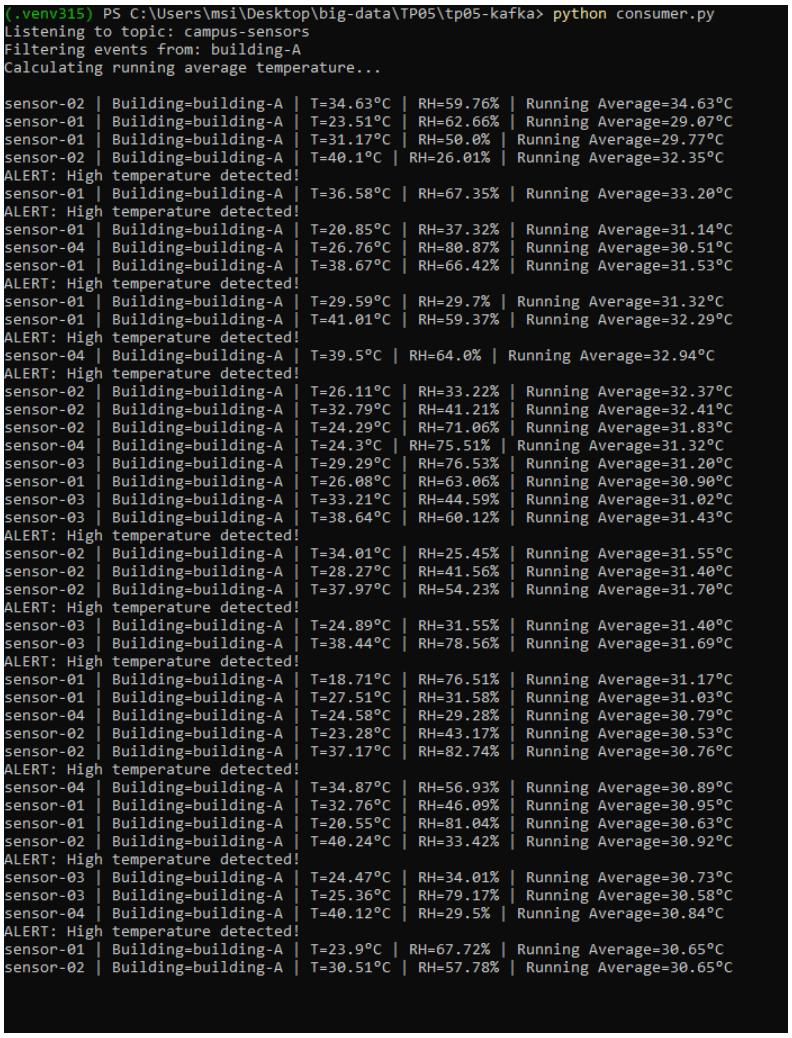


Figure 4.7: Consumer filtering events from building-A.

Chapter 5

Discussion

5.1 Analysis of the Results

The experiments demonstrate several important characteristics of Apache Kafka.

First, the producer and consumer applications communicate asynchronously through the Kafka broker. This separation allows applications to operate independently.

Second, topic partitioning enables parallel message consumption. The comparison between one partition and three partitions demonstrated the impact of partition count on the number of consumers that can actively process data within the same group.

Third, consumer groups provide two different consumption patterns. Consumers belonging to the same group share the partitions, while different groups independently read the same topic data.

Finally, the running average calculation and building-based filtering demonstrate how basic stream-processing operations can be implemented directly in a Kafka consumer.

5.2 Technical Challenges

During implementation, a compatibility issue was encountered between the tested Python environment and the `kafka-python 2.0.2` package.

A compatibility workaround using the `six` package was applied before importing Kafka classes:

```
import sys
import six

sys.modules[
```

```
    "kafka.vendor.six.moves"
] = six.moves

from kafka import KafkaProducer
```

This workaround allowed the Kafka client to run successfully in the tested environment.

5.3 Limitations

Although the implementation successfully demonstrates the main Kafka concepts, it has some limitations:

- The system uses only one Kafka broker.
- The replication factor is one, so broker-level fault tolerance is not demonstrated.
- Sensor measurements are simulated rather than collected from physical devices.
- The consumer performs basic processing rather than using a dedicated stream-processing framework.
- The system does not include a monitoring dashboard or persistent analytical database.

These limitations are acceptable for an educational experiment, but a production environment would require additional infrastructure and reliability mechanisms.

Chapter 6

Theoretical Questions and Answers

6.1 Question 1: What problem does Apache Kafka solve in a data-streaming architecture?

Apache Kafka addresses the challenge of handling and distributing continuous streams of data between applications and services.

It provides a distributed platform that allows producers to publish events and consumers to process them independently. This supports asynchronous communication, scalability, and real-time data processing.

6.2 Question 2: Define producer, consumer, broker, topic, partition, offset, and consumer group.

- **Producer:** An application that publishes messages to Kafka.
- **Consumer:** An application that subscribes to topics and reads messages.
- **Broker:** A Kafka server responsible for storing and delivering messages.
- **Topic:** A logical category used to organize messages.
- **Partition:** A subdivision of a topic that stores an ordered sequence of messages.
- **Offset:** A sequential identifier assigned to each message within a partition.
- **Consumer Group:** A group of consumers that cooperate to read messages from a topic.

6.3 Question 3: What is the difference between a Kafka topic and a partition?

A topic is a logical channel used to organize messages. A partition is a subdivision of that topic where messages are stored in an ordered sequence.

A topic can contain multiple partitions, allowing Kafka to distribute data and support parallel consumption.

In this practical work, the `campus-sensors` topic contains three partitions.

6.4 Question 4: Why does the consumer use auto_offset_rese When does this setting apply?

The `auto_offset_reset='earliest'` configuration tells the consumer to start reading from the earliest available message when it does not have a valid committed offset.

This is useful when a new consumer group starts reading a topic because it allows the consumer to process previously stored messages.

This setting does not override an existing valid committed offset.

6.5 Question 5: What is the purpose of the consumer group ID? What happens if a new group ID is used?

The consumer group ID identifies a group of consumers that cooperate to read messages from Kafka.

Consumers using the same group ID share topic partitions. Each partition is assigned to only one consumer within that group at a time.

When a new group ID is used, Kafka creates an independent consumer group with its own offset information. Therefore, different groups can independently read the same messages.

6.6 Question 6: Why is JSON a suitable format for this exercise? Give one limitation.

JSON is suitable because it is lightweight, human-readable, and easy to generate and parse using Python.

It supports structured information containing multiple fields, such as sensor ID, temperature, humidity, and timestamp.

One limitation is that JSON does not enforce a strict schema by itself. Data validation is therefore necessary to prevent inconsistent field names or data types.

6.7 Question 7: Explain how Docker Compose simplifies deployment compared with starting services manually.

Docker Compose simplifies deployment by defining services and their configurations in a single YAML file.

Instead of manually configuring and starting Kafka, the configuration specifies the container image, ports, environment variables, volumes, and restart policy.

The command `docker compose up -d` starts the configured service, making deployment easier and more reproducible.

6.8 Question 8: What are the limitations of this single-broker educational setup compared with a production Kafka cluster?

This implementation uses one Kafka broker and a replication factor of one.

Consequently, it does not provide broker-level fault tolerance or high availability. If the broker fails, the Kafka service becomes unavailable.

A production Kafka cluster generally uses multiple brokers, replicated partitions, appropriate replication factors, monitoring, security configurations, and resource management to improve reliability and scalability.

6.9 Question 9: Attach evidence showing the running container, topic description, producer output, and consumer output.

The required evidence is included in the project's `screenshots` directory.

The main screenshots are:

- `01-docker-status.png`: Running Kafka container.
- `02-topic-description.png`: Topic configuration and partitions.
- `03-producer-output.png`: Producer sending sensor events.
- `04-consumer-output.png`: Consumer receiving sensor events.
- `05-consumer-group.png`: Consumer group information.

Additional screenshots document the extended experiments and their results in the project repository.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This practical work presented an implementation of real-time data streaming and processing using Apache Kafka.

A simulated campus sensor monitoring system was developed using Python producers and consumers communicating through a Kafka broker deployed with Docker Compose.

The experiments covered topic creation, partitioning, consumer groups, message distribution, running average temperature calculation, and event filtering according to campus buildings.

The results provided practical experience with the fundamental concepts of distributed event streaming and demonstrated how Kafka can be integrated with Python applications for basic real-time data processing.

7.2 Future Work

Several improvements could extend this project:

- Deploy a multi-broker Kafka cluster.
- Increase the replication factor to improve fault tolerance.
- Connect real IoT sensors instead of simulated data.
- Integrate Apache Spark Structured Streaming for advanced processing.
- Store processed measurements in a database.

- Develop a dashboard for visualizing sensor measurements.
- Add data validation, monitoring, and alerting mechanisms.

Bibliography

[1] Apache Software Foundation, *Apache Kafka Documentation*. Available at: https://kafka.apache.org/documentation/

[2] Docker Inc., *Docker Compose Documentation*. Available at: https://docs.docker.com/compose/

[3] Python Software Foundation, *Python Documentation*. Available at: https://docs.python.org/3/

[4] Kafka Python Contributors, *kafka-python Documentation*. Available at: https://kafka-python.readthedocs.io/